

“Design and Development of AeroOps Command – An Airline Operations Briefing Dashboard for Real-Time Airport Management”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

A. KAVYA SREE :2520090143

BHAVAGNA :2520030062

Team Number: 1

Under the Esteemed Guidance of

Ms. D. Pavitra

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
&
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Declaration

The Project Report entitled “Design and Development of AeroOps Command – An Airline Operations Briefing Dashboard for Real-Time Airport Management” is a record of bonafide work of A. KAVYA SREE - 2520090143, BHAVAGNA - 2520030062 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

A. KAVYA SREE – 2520090143
BHAVAGNA – 2520030062

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
&
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the mini project based report entitled “Design and Development of AeroOps Command – An Airline Operations Briefing Dashboard for Real-Time Airport Management” is a bonafide work done and submitted by A. KAVYA SREE - 2520090143, BHAVAGNA - 2520030062 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year 2025-2026.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who assisted us in the completion of this project.

Our greatest appreciation goes to our guide, Ms. D. Pavitra, Assistant Professor, Department of Computer Science and Engineering, whose tremendous support, encouragement, and guidance made this project possible.

We express our gratitude to the Head of the Department, Department of Computer Science and Engineering, for providing us with adequate facilities, ways, and means to complete this project-based work.

We thank all members of the teaching and non-teaching staff who assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends, and classmates for their kind help and cooperation throughout our work.

A. KAVYA SREE – 2520090143

BHAVAGNA – 2520030062

Abstract

Airline operations involve managing multiple activities such as flight scheduling, slot allocation, delay monitoring, staff coordination, weather tracking, and gate management. Handling these activities manually can lead to operational inefficiencies, delays, and poor decision-making. To address these challenges, this project presents AeroOps Command – Airline Operations Briefing Dashboard, a web-based application designed to provide airport managers with a centralized platform for monitoring daily airport operations.

The dashboard presents real-time operational insights through various modules including KPI Summary, Slot Utilization Analysis, Delay Heatmap, Weather Monitoring, Staff Management, Gate Conflict Alerts, Flight Departure Board, Notifications, Export Briefing, and Admin Controls. The system enables users to visualize critical airport data, identify operational bottlenecks, monitor flight delays, and respond quickly to conflicts affecting airport efficiency. The application is developed using React.js, TypeScript, HTML5, CSS3, and JavaScript (ES6+), following a component-based architecture and modern front-end engineering principles. Features such as dynamic data generation, interactive dashboards, responsive design, and state management provide an efficient user experience. The system also supports date-based operational analysis, allowing users to view different airport scenarios for different days.

The proposed solution improves operational awareness, supports data-driven decision-making, enhances resource utilization, and provides a scalable foundation for future integration with real-time airline and airport management systems.

Index

S.No.	Chapter	Topics	Page No.
1	Introduction	1.1 Background of the project, 1.2 Problem statement, 1.3 Scope, 1.4 Objectives, 1.5 Importance, 1.6 Target users	5-15
2	System Requirements	2.1 Hardware, 2.2 Software, 2.3 Development tools & frameworks	16-21
3	Technology Stack	3.1 Front-end, 3.2 Back-end, 3.3 Database, 3.4 Version Control, 3.5 APIs/External services	22-32
4	System Architecture	4.1 High-level architecture, 4.2 Layer description, 4.3 Deployment architecture	33-36
5	Design	5.2 ER Diagram / Database schema	37
6	Implementation	6.1 Module-wise, 6.2 Front-end logic, 6.3 API integration, 6.4 Auth & Authorization	38-47
7	Features	7.1 List of features, 7.2 How each feature works	48-55
8	Testing	8.1 Postman testing, 8.2 Integration testing, 8.3 Tools used, 8.4 Test cases & results	56-64
9	Deployment	9.1 Steps to deploy, 9.2 Environment config, 9.3 Hosting	65-73
10	Challenges & Limitations	10.1 Issues faced, 10.2 Solutions applied, 10.3 Current limitations	74-82
11	Future Enhancements	11.1 Planned features, 11.2 Possible integrations/optimizations	83-88

S.No.	Chapter	Topics	Page No.
12	Conclusion	12.1 Summary, 12.2 What was achieved, 12.3 Skills learned	89-97
13	References	Books, tutorials, APIs, documentation sites used	98-99
14	Appendices	Screenshots, sample code, installation guide, user manual	100-105

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

The aviation industry relies heavily on efficient operational management to ensure smooth airport functioning, passenger satisfaction, and flight safety. Every day, airport managers must monitor multiple operational factors such as flight schedules, slot utilization, delays, weather conditions, gate assignments, and staff availability. Managing this information through separate systems can be time-consuming and may lead to delayed decision-making during critical situations.

Historically, airport operations centres have relied on a combination of radio communication, printed schedules, and disconnected legacy software to track day-to-day activity. As passenger traffic and flight frequency have grown, this fragmented approach has become increasingly difficult to sustain. The emergence of modern front-end web technologies, capable of rendering rich, interactive dashboards in real time, has created an opportunity to consolidate these disparate operational feeds into a single, coherent interface.

AeroOps Command was conceived against this backdrop – as a centralized, web-based operations briefing dashboard that draws together the key indicators an airport duty manager needs at a glance. The project builds on component-based front-end engineering practices learned during coursework, applying them to a realistic, data-intensive domain problem.

1.2 Problem Statement

Airports handle a large number of daily operations, including flight scheduling, slot allocation, gate management, staff coordination, delay monitoring, and weather assessment. These operational activities generate significant amounts of data that must be monitored continuously to ensure smooth airport functioning. However, information is often distributed across multiple systems, making it difficult for airport managers to obtain a unified view of ongoing operations.

The absence of a centralized monitoring platform can lead to delayed decision-making, inefficient resource utilization, increased flight delays, gate conflicts, staffing issues, and reduced operational efficiency. Airport managers require a system that can consolidate critical operational information and present it in an easy-to-understand format for rapid analysis and response.

To address this challenge, there is a need for a web-based Airport Operations Briefing Dashboard that integrates key operational metrics such as flight status, slot utilization, delay analysis, weather conditions, staffing information, gate conflict alerts, and administrative controls into a single platform. The system should provide real-time insights, interactive visualizations, notifications, and reporting capabilities to support efficient airport management and improve overall operational performance.

1.3 Scope of the Project

The scope of AeroOps Command is limited to the design and development of a front-end-driven operations briefing dashboard intended for use by airport duty managers and operations staff. The system covers ten functional modules: KPI Summary, Slot Utilization Analysis,

Delay Heatmap, Weather Monitoring, Staff Management, Gate Conflict Alerts, Flight Departure Board, Notifications, Export Briefing, and Admin Controls, together with a secure Login page.

Within this scope, the platform supports date-based scenario switching, allowing a user to review the operational picture for a chosen day, and it integrates one live external data source (the OpenWeather API) to demonstrate real-world API consumption, while other operational feeds are represented using structured, dynamically generated datasets that mirror realistic airport data patterns.

The project does not extend into live flight-booking, air-traffic-control, or safety-critical systems, nor does it interface with real airline reservation or radar systems; such integrations are identified as future enhancements. The present scope is therefore an operational-briefing and decision-support layer that sits above (and could, in future, be wired into) an airline's live operational data sources.

1.4 Objective(s)

1. To develop a centralized airport operations dashboard. Justification: A centralized platform allows airport managers to monitor all critical operational information from a single interface, improving efficiency and decision-making.
2. To monitor flight operations and delays in real time. Justification: Real-time tracking of flights and delays helps identify operational issues quickly and supports timely corrective actions.
3. To analyze airport slot utilization and resource allocation. Justification: Efficient slot management reduces congestion, improves airport capacity utilization, and supports smooth flight operations.
4. To detect and manage gate conflicts effectively. Justification: Early detection of gate conflicts minimizes operational disruptions and helps maintain scheduled flight movements.
5. To provide operational alerts, reports, and decision-support information. Justification: Notifications, alerts, and exportable reports enable airport managers to respond quickly to issues and make data-driven decisions.

1.5 Importance of the Application

AeroOps Command demonstrates how a well-designed front-end can transform scattered operational data into an actionable briefing tool. By surfacing KPI summaries, delay heatmaps, and gate conflict alerts on a single screen, the application shortens the time between an operational event occurring and a manager becoming aware of it – directly supporting faster, better-informed decisions.

Beyond its immediate operational value, the project is an important academic exercise. It demonstrates the practical application of component-based architecture, state management, responsive design, and third-party API integration to a genuinely complex, real-world domain, and it gives the team hands-on experience with the kind of dashboard-style product commonly built in industry (in aviation, logistics, finance, and operations-heavy sectors generally).

Finally, the modular design means the dashboard can act as a scalable foundation: individual modules can be progressively connected to live airline and airport data feeds without requiring a redesign of the overall interface.

1.6 Target Users/Audience

The primary users of AeroOps Command are airport duty managers and operations-centre staff who need a consolidated, real-time view of airport activity to coordinate flights, gates, and personnel.

Secondary users include airline station managers who need visibility into slot utilization and delay patterns affecting their flights, ground-staff supervisors who rely on the staff management module to plan shifts and availability, and administrative personnel who use the admin panel to configure system settings and generate operational reports.

The project also serves an academic audience – faculty evaluators and fellow students – as a demonstration of applied front-end engineering, and it can serve as a portfolio project for the development team when presenting their technical capabilities to prospective employers.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Development of AeroOps Command does not demand specialized or high-end hardware, since the application is a client-side, browser-rendered dashboard. The project was developed and tested on a standard laptop configuration, summarized below alongside the recommended minimum for smooth development.

Component	Minimum Requirement	Used for Development
Processor	Dual-core, 1.6 GHz or higher	11th Gen Intel Core i5
RAM	4 GB	8 GB
Storage	5 GB free disk space	SSD, 8 GB free space used
Graphics	Integrated graphics sufficient	Integrated graphics
Display	1366 × 768 or higher	1920 × 1080
Internet	Broadband connection	Broadband connection

2.2 Software Requirements

The software environment consists of a modern operating system, a code editor, a current-generation web browser for testing, and the Node.js runtime required to run the Vite development server and build tooling.

Category	Software Used
Operating System	Windows 10/11 (also compatible with macOS/Linux)
Code Editor	Visual Studio Code
Browser	Google Chrome (primary), Microsoft Edge, Firefox (cross-browser testing)
Runtime	Node.js (LTS)
Package Manager	npm
Version Control	Git, hosted on GitHub

2.3 Development Tools and Frameworks

React.js was chosen as the core UI library for its component-based architecture and efficient virtual-DOM rendering. TypeScript was layered on top of React to add static typing, catching type-related errors during development rather than at runtime. Vite was used as the build tool and development server for its fast hot-module-reload and optimized production builds.

Tailwind CSS provided a utility-first styling approach that kept the interface visually consistent while allowing rapid iteration on layout and responsiveness. React Router was used to manage client-side navigation between the dashboard's pages. Axios handled HTTP requests to the

OpenWeather API. Postman was used during development to test and validate API responses independently of the front-end code, and Git/GitHub were used throughout for version control and collaboration between the two team members.

CHAPTER -3 TECHNOLOGY STACK

3.1 Front-end

The front end of AeroOps Command is built entirely with React.js and TypeScript, using functional components and React Hooks (useState, useEffect, useMemo, useContext) for state and lifecycle management. TypeScript's static typing was particularly valuable for a data-heavy dashboard: interfaces were defined for every core entity (Flight, Gate, Slot, StaffMember, WeatherRecord, Notification) so that the shape of data flowing between modules is checked at compile time.

Styling is handled with Tailwind CSS, whose utility classes made it possible to build a consistent design language – spacing, colour palette, typography, and responsive breakpoints – without writing large volumes of custom CSS. HTML5 semantic elements and CSS3 features (flexbox, grid, transitions) round out the presentation layer, and JavaScript ES6+ syntax (arrow functions, destructuring, async/await, template literals) is used throughout for cleaner, more maintainable code.

React Router manages navigation between the Login page and the ten dashboard modules, and the overall build is bundled and served by Vite, which was chosen over Create React App for its significantly faster development server and smaller production bundle size.

3.2 Back-end

AeroOps Command is, by design, a front-end-driven prototype. Rather than depending on a persistent server-side application, most operational data (flights, slots, gates, staff rosters) is produced by structured, dynamic data-generation modules written in TypeScript that simulate realistic day-to-day airport activity, including variation by the date selected on the dashboard.

This decision kept the project scope achievable within the academic timeline while still allowing every dashboard module to behave as if it were backed by a live operational feed. The service layer is intentionally structured so that each data generator can be swapped for a genuine REST/GraphQL endpoint (for example, a Node.js and Express API backed by a relational database) with minimal change to the React components that consume it – this extensibility is discussed further in Chapter 11 (Future Enhancements).

3.3 Database

As the current release does not persist data server-side, there is no live production database. Instead, a conceptual data model (see Chapter 5, Section 5.2) defines the entities – Flight, Gate, Slot, Staff, WeatherRecord, Notification, Report, and AdminUser – and their relationships, mirroring the schema that a relational database (such as MySQL or PostgreSQL) would use if the system were connected to a persistent back end.

Designing this schema up front ensures that when the project moves to a database-backed implementation, the existing TypeScript interfaces and component props can be mapped directly onto database tables with minimal refactoring.

3.4 Version Control

Git was used for version control throughout development, with the repository hosted on GitHub. The team followed a feature-branch workflow: new modules and fixes were developed on dedicated branches and merged into the main branch through pull requests, which allowed each change to be reviewed before integration.

Commit messages followed a consistent, descriptive convention (e.g., “feat: add delay heatmap component”, “fix: correct gate conflict detection logic”) to keep the project history readable, and GitHub Issues were used informally to track outstanding tasks and bugs during the review cycle.

3.5 APIs and External Services

The OpenWeather API is the one external live data source integrated into the system, powering the Weather Monitoring module with current conditions, temperature, and wind-speed data for the airport's location. Axios is used as the HTTP client for this integration, with request/response interceptors handling loading states and error fallbacks so that a temporary API failure does not break the rest of the dashboard.

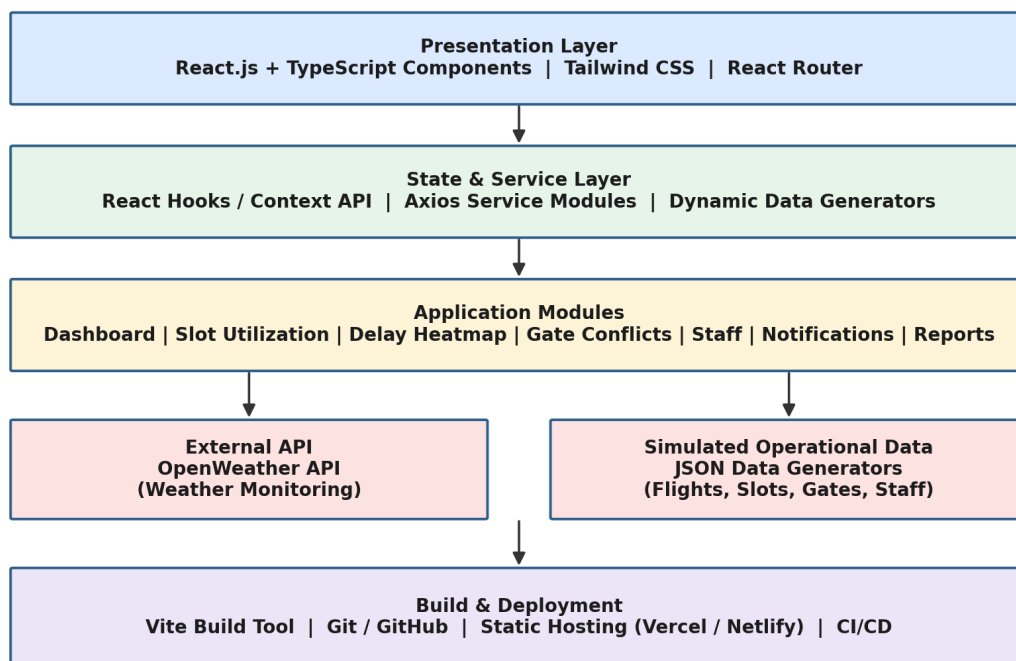
All other modules currently rely on the internal dynamic data-generation layer described in Section 3.2, which was designed to be API-shaped – that is, it mimics the request/response and typing conventions of a real REST API, so that a genuine airline operations API could be substituted in with minimal disruption to the UI layer.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The architecture of AeroOps Command is organized into four broad layers: a presentation layer built from React/TypeScript components, a state and service layer that manages application state and mediates data access, an application-modules layer representing the ten functional dashboard pages, and a build/deployment layer. The diagram below summarizes how these layers relate to one another.

Fig. 4.1 High-Level System Architecture of AeroOps Command



4.2 Description of Each Layer

Presentation Layer: This is the topmost layer, comprising reusable React components – KPI cards, charts, tables, alert banners, and navigation elements – styled with Tailwind CSS and routed with React Router. Components are kept small and composable so that the same building blocks (for example, a generic “StatCard” or “AlertBanner”) can be reused across multiple modules.

State and Service Layer: Below the presentation layer sits the logic that manages application state (via React Hooks and Context API where state needs to be shared across components, such as the currently selected date) and the service modules that fetch data – either from the OpenWeather API via Axios, or from the internal dynamic data generators.

Application Modules Layer: This layer represents the ten functional pages of the dashboard – Dashboard/KPI Summary, Slot Utilization, Delay Heatmap, Flight Operations, Weather Monitoring, Gate Conflict, Staff Management, Notifications, Admin Panel, and Export Reports – each composed from the shared component library and connected to the service layer.

Build and Deployment Layer: At the base of the stack, Vite compiles and bundles the TypeScript/React source into optimized static assets, Git/GitHub manage source control, and the resulting build is deployed to a static hosting platform, with a CI/CD pipeline automating the build-and-deploy sequence on every push to the main branch.

4.3 Deployment Architecture

Because AeroOps Command is a purely static, client-rendered application once built, it is well suited to modern static-hosting platforms (such as Vercel or Netlify) that serve the compiled assets from a global content-delivery network (CDN), giving users fast load times regardless of their geographic location.

A simple CI/CD pipeline, configured through GitHub Actions, automatically runs a type-check and build on every pull request, and triggers a production deployment when changes are merged into the main branch. This automated pipeline reduces the risk of shipping a broken build and keeps the deployed dashboard continuously in sync with the reviewed source code.

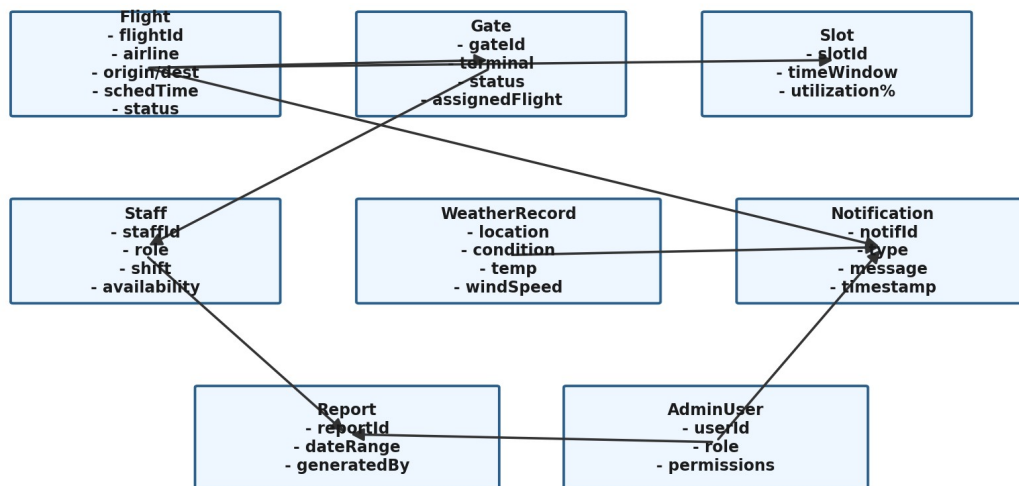
CHAPTER -5 DESIGN

5.2 ER Diagram / Database Schema

Although the current implementation does not persist data to a live database, the system was designed around the conceptual entity-relationship model shown below, which reflects the data structures actually used within the TypeScript codebase and anticipates the schema that would back a future, database-connected version of the platform.

Flight is the central entity, related to Gate (through gate assignment) and Slot (through scheduled time-slot allocation). Staff records are associated with Gates for duty assignment. WeatherRecord data feeds into Notification generation (for example, a severe-weather alert). Report entities aggregate data generated by AdminUser actions, and AdminUser also has authority to trigger Notifications.

Fig. 5.1 Conceptual Entity Relationship Diagram



CHAPTER -6 IMPLEMENTATION

6.1 Module-Wise Implementation

AeroOps Command is implemented as eleven distinct pages/modules, each built as an independent set of React components sharing a common design system. The table below summarizes each module and its core responsibility.

Module	Description
Login Page	User authentication and secure access
Dashboard Page	Displays KPI summaries and airport overview
Slot Utilization Page	Monitors airport slot allocation
Delay Analysis Page	Shows flight delay heatmaps
Flight Operations Page	Displays flight schedules and status
Weather Monitoring Page	Shows weather information
Gate Conflict Page	Detects gate assignment conflicts
Staff Management Page	Manages staff schedules and availability
Notifications Page	Displays alerts and operational updates
Admin Panel	System settings and configuration
Export Reports Page	Generates operational reports

6.2 Front-End Logic (UI Rendering, State Management)

Each page is composed from smaller, reusable components (cards, tables, charts, alert badges) so that visual consistency is maintained across the dashboard without duplicating code. Local component state (e.g., a filter selection or a toggled panel) is managed with `useState`, while state that needs to be shared across modules – most importantly, the globally selected date that drives the “date-based operational analysis” feature – is lifted into a React Context provider at the application root.

`useEffect` hooks manage side effects such as re-fetching weather data when the selected date or location changes, and `useMemo` is used to avoid recomputing derived values (for example, aggregated KPI totals or heatmap intensity buckets) on every render. Conditional rendering and skeleton/loading states keep the interface responsive even while data is being generated or fetched.

6.3 API Integration

API integration is centralized in a small set of service modules. The `weatherService` module wraps `Axios` calls to the OpenWeather API, normalizing the response into the application's internal `WeatherRecord` type and handling error and timeout cases gracefully with a cached fallback value. The remaining service modules (`flightService`, `slotService`, `gateService`, `staffService`, `notificationService`) present an equivalent async, Promise-based interface, but are

backed internally by the dynamic data generators described in Chapter 3 – keeping the calling code in each page component identical regardless of whether the underlying data source is a live API or a simulated one.

6.4 Authentication & Authorization

The Login page implements a straightforward credential-based authentication flow appropriate to the project's academic scope: a user submits credentials, which are validated against a stored/mock user record, and on success a session flag is set (using React state/Context) that gates access to the protected dashboard routes via React Router's route-guarding pattern.

The Admin Panel is further restricted to users with an administrative role, demonstrating a simple role-based authorization check on top of the base authentication flow. While this implementation is intentionally lightweight for the current prototype, the authentication module is structured so that it could be swapped for a token-based (JWT) scheme backed by a real authentication service without altering the surrounding route-guard logic.

CHAPTER -7 FEATURES

7.1 List of Main Features

- KPI Summary cards giving an at-a-glance overview of daily airport performance (on-time percentage, active flights, open alerts, gate utilization).
- Slot Utilization analysis showing how efficiently allocated time slots are being used across the day.
- Delay Heatmap visualizing flight delays across time bands and routes to highlight recurring bottlenecks.
- Flight Departure Board listing scheduled, delayed, boarding, and departed flights in real time.
- Weather Monitoring panel showing live conditions sourced from the OpenWeather API.
- Gate Conflict Alerts that automatically flag overlapping gate assignments.
- Staff Management view for tracking shift schedules and personnel availability.
- Notifications centre aggregating operational alerts across all modules.
- Export Briefing tool for generating a shareable operational report.
- Admin Controls for configuring system settings and managing users.
- Date-based scenario switching, letting a user review any day's operational picture.

7.2 How Each Feature Works (User Perspective + Technical Description)

User Perspective: A duty manager logs in and lands on the Dashboard, where KPI cards immediately summarize the day's operational health. From there, they can drill into Slot Utilization or the Delay Heatmap to investigate a specific concern, check the Weather Monitoring panel before making a gate-reassignment decision, review the Gate Conflict page if an alert has fired, and finally use Export Briefing to generate a report for a shift handover.

Technical Description: Each feature is implemented as a page-level component that requests data from its corresponding service module on mount (and whenever the selected date changes), transforms the raw data into the shapes needed by presentational sub-components (cards, charts, tables), and renders those sub-components inside a shared page layout. The Gate Conflict detection logic, for example, iterates over the day's gate assignments and flags any two flights whose scheduled ground-time windows at the same gate overlap, surfacing the result both on the Gate Conflict page and as an entry in the global Notifications centre.

The Delay Heatmap groups flight delay minutes into hourly bands and colour-codes each cell by severity, giving an immediate visual signal of when delays cluster during the day. The Export Briefing feature compiles the current date's KPI, delay, and alert data into a formatted summary that the user can download, supporting quick handovers between operational shifts.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

Postman was used to independently verify the OpenWeather API integration before it was wired into the front end – confirming request parameters, response structure, and error codes for invalid locations or missing API keys. This allowed the weatherService module to be written against a known, verified contract.

8.2 Integration Testing

Integration testing focused on verifying that data flowed correctly from each service module through to its corresponding page component: that a change in the globally selected date correctly re-triggered every dependent module's data refresh, that the Gate Conflict page's alerts appeared correctly in the Notifications centre, and that loading and error states rendered as expected when a simulated API delay or failure was introduced.

8.3 Tools Used for Testing

Testing combined manual, exploratory testing in Chrome DevTools (including the responsive-design mode, to verify behaviour across desktop, tablet, and mobile breakpoints), Postman for API-contract verification, and React Testing Library for targeted component-level tests of key logic such as the gate-conflict detection function and the delay-heatmap bucketing logic.

8.4 Test Cases and Results

A representative sample of the test cases executed against the system is summarized below.

Test Case	Steps / Input	Expected Result	Status
Login – valid credentials	Enter valid username & password	User redirected to Dashboard	Pass
Login – invalid credentials	Enter incorrect password	Error message displayed, access denied	Pass
Date switch updates KPIs	Select a different date on Dashboard	All KPI cards refresh with that date's data	Pass
Gate conflict detection	Assign two flights to the same gate with overlapping times	Conflict alert generated and shown in Notifications	Pass
Weather API failure fallback	Simulate OpenWeather API timeout	Cached/fallback weather message shown, no crash	Pass
Delay heatmap rendering	Load Delay Analysis page with sample delay data	Heatmap cells colour-coded by delay severity	Pass
Export briefing	Click Export on current date	Downloadable report generated with correct	Pass

Test Case	Steps / Input	Expected Result	Status
		data	
Responsive layout	Resize viewport to mobile width	Layout reflows to single-column, no overflow	Pass
Admin-only access	Access Admin Panel as non-admin user	Access denied / redirected	Pass

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

1. Run the production build with Vite (`npm run build`), which type-checks the TypeScript source and outputs optimized, minified static assets.
2. Push the reviewed, merged code on the main branch to GitHub, triggering the CI/CD pipeline.
3. The pipeline installs dependencies, runs the build step, and, on success, publishes the generated assets to the static hosting platform.
4. The hosting platform serves the build from its CDN, and the OpenWeather API key is supplied to the running application via an environment variable configured on the hosting dashboard rather than committed to source control.

9.2 Environment Configuration

Configuration values that differ between local development and production – principally the OpenWeather API key and the base URL used by the service layer – are managed through Vite's environment-variable system (`.env` files locally, and the hosting platform's secret/environment configuration in production), keeping sensitive keys out of the committed codebase.

9.3 Hosting of Frontend and Backend

As AeroOps Command is currently a front-end-only application (see Chapter 3), only the static build needs to be hosted, which was done using a modern static-hosting provider offering automatic CDN distribution, HTTPS, and preview deployments for each pull request. This setup was chosen deliberately so that, when a genuine back-end service is introduced in a future iteration, it can be hosted independently (for example, on a managed Node.js platform) without requiring any change to how the front end itself is deployed.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

Designing dynamic data generators that produced realistic, internally consistent airport data (for example, ensuring that a flight's delay status, gate assignment, and notification entries all agreed with one another) proved more intricate than initially expected, since inconsistent mock data made several components difficult to test in isolation.

Coordinating shared state – particularly the globally selected date – across ten independently developed modules occasionally led to stale or out-of-sync data appearing in one module after a date change in another, before the team settled on a single Context-based source of truth.

Working within the free tier of the OpenWeather API introduced rate-limiting constraints that had to be accounted for during development and testing.

10.2 Solutions Applied

The team centralized all date-dependent state in a single React Context provider at the application root, ensuring every module subscribes to the same source of truth and re-renders consistently when the date changes.

A shared, deterministic data-generation seed (tied to the selected date) was introduced so that all modules referencing the same date produce mutually consistent flight, gate, and delay data, eliminating the earlier inconsistencies.

A lightweight caching layer was added around the weatherService calls to reduce the number of live OpenWeather API requests and stay comfortably within the free-tier rate limits.

10.3 Current Limitations or Known Bugs

The system does not currently persist data between sessions, since there is no connected database – refreshing the browser resets any admin-configured settings to their defaults.

Because operational data (other than weather) is simulated rather than sourced from a live airline system, the dashboard demonstrates realistic behaviour but does not yet reflect actual flight operations.

The authentication mechanism, while functionally adequate for demonstration purposes, is not production-grade and would need to be replaced with a proper token-based scheme (see Chapter 11) before any real-world deployment.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

A planned enhancement is the introduction of an AI-based disruption-prediction module that would use historical delay patterns to forecast the likelihood of upcoming disruptions, giving managers advance warning rather than purely reactive alerts.

The team also plans to extend the Export Briefing module to support scheduled, automatic report generation (e.g., an end-of-shift email summary) and to add a customizable KPI dashboard that lets each user choose which metrics appear on their landing page.

11.2 Possible Integrations or Optimizations

The most significant future integration is replacing the internal dynamic data generators with a genuine Node.js/Express back end backed by a relational database (MySQL or PostgreSQL), implementing the conceptual schema from Chapter 5, so that the dashboard reflects real, persisted operational data.

Token-based (JWT) authentication with proper role-based access control would replace the current lightweight login flow, and WebSocket-based live updates could replace polling for near-instant propagation of flight-status changes and new alerts across all connected clients.

On the performance side, code-splitting and lazy-loading of individual dashboard modules would be introduced to keep initial load times low as the feature set grows, and integration with additional external data sources (a live flight-status API, in particular) would move the Flight Operations and Delay Heatmap modules from simulated to fully live data.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

AeroOps Command – Airline Operations Briefing Dashboard was developed to provide a centralized platform for monitoring and managing airport operations. The system integrates key operational modules such as flight monitoring, slot utilization analysis, delay tracking, weather monitoring, gate conflict detection, staff management, notifications, and reporting. By presenting operational data through an interactive and user-friendly dashboard, the system helps airport managers improve decision-making, resource utilization, and operational efficiency.

12.2 What Was Achieved

The team successfully delivered a fully functional, responsive dashboard covering all eleven planned modules, implemented in React.js and TypeScript with a consistent Tailwind CSS design system. A live external API integration (OpenWeather) was completed, alongside a realistic internal data-simulation layer for the remaining operational feeds, and the date-based scenario-switching feature was implemented and verified across all modules.

The project demonstrates the effective use of modern front-end technologies to address real-world challenges in the airline and airport management domain, and it was built, tested, and deployed following a disciplined Git-based workflow with automated CI/CD.

12.3 Skills Learned During Development

Working on AeroOps Command strengthened the team's practical skills in component-based front-end architecture, TypeScript typing discipline, and state-management patterns (Hooks and Context) applied to a genuinely data-heavy application.

The team also gained direct experience integrating and gracefully degrading around a live third-party API, structuring a codebase so that simulated data and live data can be swapped transparently, and using Git branching, pull requests, and CI/CD pipelines as part of a real collaborative workflow – skills directly transferable to industry front-end engineering roles.

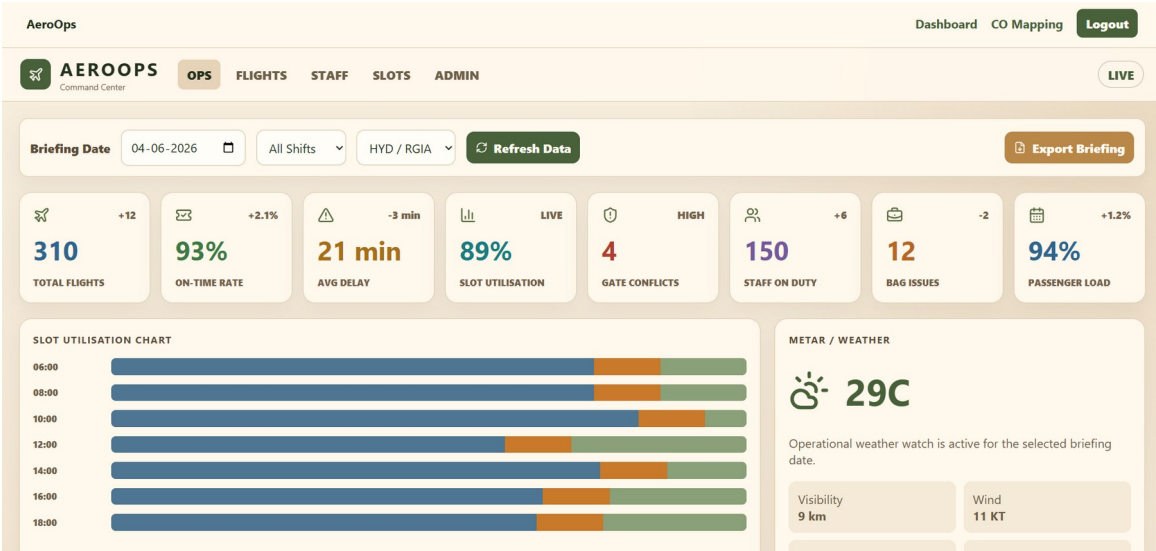
CHAPTER -13 REFERENCES

1. React.js Documentation. Available at: <https://react.dev/>
2. TypeScript Documentation. Available at: <https://www.typescriptlang.org/docs/>
3. Vite Documentation. Available at: <https://vitejs.dev/>
4. MDN Web Docs – HTML, CSS and JavaScript. Available at: <https://developer.mozilla.org/>
5. React Router Documentation. Available at: <https://reactrouter.com/>
6. Tailwind CSS Documentation. Available at: <https://tailwindcss.com/docs>
7. Axios Documentation. Available at: <https://axios-http.com/docs/intro>
8. OpenWeather API Documentation. Available at: <https://openweathermap.org/api>
9. Federal Aviation Administration (FAA). Airport Operations Guidelines. Available at: <https://www.faa.gov/>
10. International Air Transport Association (IATA). Airport and Airline Operations Resources. Available at: <https://www.iata.org/>

CHAPTER -14 APPENDICES

14.1 Screenshots of the Application

Representative screenshots of the AeroOps Command dashboard interface are shown below, illustrating the KPI summary view and supporting visual elements referenced in Chapter 8.



A

AEROOPS

Airport Operations Command Center

Operator ID

admin

Security Passcode

admin123

Access Command Center

14.2 Sample Code Snippet

```
function detectGateConflicts(assignments) {    const conflicts = [];    for
(let i = 0; i < assignments.length; i++) {        for (let j = i + 1; j <
assignments.length; j++) {            if (assignments[i].gateId ===
assignments[j].gateId && overlaps(assignments[i].window,
assignments[j].window)) {                conflicts.push([assignments[i],
assignments[j]]);            }        }    }    return conflicts; }
```

14.3 Installation / Setup Instructions

1. Clone the repository: `git clone <repository-url>`
2. Install dependencies: `npm install`
3. Create a `.env` file with the OpenWeather API key:
`VITE_OPENWEATHER_API_KEY=<your-key>`
4. Start the development server: `npm run dev`
5. Build for production: `npm run build`

14.4 User Manual / Guide

On launching the application, the user logs in via the Login page. Upon successful authentication, the user is directed to the Dashboard, which serves as the entry point to all other modules accessible through the navigation menu. Selecting a date from the date picker updates all data-driven modules to reflect that day's operational scenario.